



摘要

分子动力学 (Molecular Dynamics, MD) 模拟是研究生物大分子动态行为的重要计算工具, 被誉为“计算显微镜”。随着全细胞模拟目标的提出, 传统基于快速傅里叶变换 (Fast Fourier Transform, FFT) 的长程静电求解方法——如粒子网格 Ewald (Particle Mesh Ewald, PME) 和粒子-粒子-粒子-网格 (Particle-Particle Particle-Mesh, PPPM) ——在大规模并行计算中面临严重的通信瓶颈。本研究针对这一关键问题, 提出了一种基于高斯和分解 (Sum-of-Gaussians, SOG) 与随机批量采样 (Random Batch Sampling) 相结合的新型长程静电求解框架, 即基于随机批量采样与高斯和分解的长程静电方法 (Random Batch Sum-of-Gaussians, RBSOG)。

本研究构建了面向膜体系的分子动力学模拟原型系统, 实现了三种长程静电求解器: 直接求和参考解 ($O(N^2)$)、PPPM 基线方法和 RBSOG 方法。通过系统化的基准测试, 从计算性能、统计方差和结构稳定性三个维度对 PPPM 和 RBSOG 进行了全面比较。基于本文重新串行执行的真实实验结果, RBSOG 在当前纯 Python 原型中的平均每步时间显著高于 PPPM, 在 10 个随机种子统计下约为其 5.468 倍; 同时, RBSOG 的压力方差均值也高于 PPPM, 说明该方法在当前实现阶段尚未展现出统计稳定性优势。不过, 批量大小扫描与 JIT 消融实验表明, RBSOG 对实现细节和参数选择较为敏感, 且 JIT 加速可带来约 2.322 倍的内核级提速, 这表明该方法仍具有进一步工程优化空间。

本研究的主要贡献包括: (1) 提出了 RBSOG 长程静电求解框架, 将高斯和分解与随机批量采样相结合, 为大规模分子动力学模拟提供了新的算法思路; (2) 建立了兼顾计算效率、统计方差和结构稳定性的综合评测体系; (3) 构建了可复现的自动化实验与论文生成流程; (4) 通过系统化实验分析, 明确给出了当前原型阶段 RBSOG 与 PPPM 之间的真实性能差距与优化方向。研究结果表明, RBSOG 方法在当前实现下尚未优于 PPPM, 但具有进一步工程优化和向更大规模体系迁移的潜力, 为未来全细胞分子动力学模拟中的长程静电求解提供了新的技术路径。

关键词: 分子动力学模拟, 长程静电, 高斯和分解, 随机批量采样, 全细胞模拟, 膜体系



Abstract

Molecular Dynamics (MD) simulation is an important computational tool for studying the dynamic behavior of biological macromolecules, known as the “computational microscope.” With the advent of whole-cell simulation goals, traditional long-range electrostatics solvers based on the Fast Fourier Transform (FFT)—such as Particle Mesh Ewald (PME) and Particle-Particle Particle-Mesh (PPPM)—face severe communication bottlenecks in large-scale parallel computing. This study addresses this critical problem by proposing a novel long-range electrostatics framework, Random Batch Sum-of-Gaussians (RBSOG), which combines Sum-of-Gaussians (SOG) decomposition with random batch sampling.

We built a membrane-oriented molecular dynamics prototype with three long-range electrostatics solvers: direct pairwise Coulomb reference ($O(N^2)$), the PPPM baseline, and the proposed RBSOG method. Through systematic benchmarking, we compared PPPM and RBSOG across computational performance, statistical variance, and structural stability. Based on the final serially executed experiments in this thesis, RBSOG remains about 5.468 times slower than PPPM in the current pure-Python prototype and also shows higher pressure variance in the 10-seed benchmark. Nevertheless, batch-size sweep and just-in-time compilation ablation results indicate that the method is highly sensitive to implementation details and still has room for optimization. Furthermore, JIT acceleration provides about 2.322 times kernel-level speedup in the current implementation, and we introduced a weighted Pareto frontier analysis method for automated batch size parameter P recommendation.

The main contributions of this study include: (1) proposing the RBSOG long-range electrostatics framework combining SOG decomposition with random batch sampling, providing a new algorithmic approach for large-scale MD simulation; (2) establishing a comprehensive evaluation system balancing computational efficiency, statistical variance, and structural stability; (3) constructing a reproducible automated experiment and paper generation pipeline; (4) identifying, through real benchmark evidence, the current performance gap and optimization priorities of the prototype. The results indicate that RBSOG has potential for further engineering optimization and migration to larger systems, provid-



ing a new technical pathway for long-range electrostatics in future whole-cell molecular dynamics simulations.

Keywords: Molecular Dynamics, Long-Range Electrostatics, Sum-of-Gaussians, Random Batch Sampling, Whole-Cell Simulation, Membrane System



目 录

第一章 引言	1
1.1 研究背景与意义	1
1.2 长程静电求解的挑战与机遇	2
1.3 研究目标与内容	3
第二章 方法	5
2.1 分子动力学基础理论	5
2.2 RBSOG 算法框架	6
2.3 膜体系构建方法	7
2.4 评测指标设计	9
2.5 实验环境与配置	10
2.6 PPPM 方法详解	11
2.7 直接求和参考解	13
2.8 RBSOG 方法的详细实现	13
2.9 实验设计	14
第三章 结果	16
3.1 PPPM 与 RBSOG 主基准测试	16
3.2 批量大小扫描实验	18
3.3 结构稳定性分析	20
3.4 JIT 消融实验	21
3.5 十种子扩展统计实验	22
3.6 本章小结	23
第四章 讨论	24
4.1 结果解释与机理分析	24
4.2 与现有工作的比较	25
4.3 创新点与局限性	25
4.4 未来工作展望	26
第五章 结论	28
5.1 主要工作总结	28
5.2 创新点总结	28
5.3 贡献与意义	29
5.4 最终结论	29



附录 A 实现补充与复现实验说明	30
A.1 真实实现说明	30
A.2 RBSOG 求解器真实核心代码节选	30
A.3 近邻表与短程求解真实实现节选	33
A.4 SOG 核拟合真实实现节选	34
A.5 复现实验命令	35
A.6 附录小结	37
参考文献	38
致谢	39



第一章 引言

1.1 研究背景与意义

计算显微镜：跨越时空尺度的生物学新范式

在过去半个世纪中，结构生物学经历了从 X 射线晶体学到冷冻电子显微镜 (Cryogenic Electron Microscopy, Cryo-EM) 的革命性飞跃，使得人类能够以原子级分辨率解析单个蛋白质及其复合物的静态结构。然而，生命现象的本质在于”动态”与”相互作用”。细胞内部并非稀溶液环境，而是一个拥挤、异质且高度动态的软物质系统。大分子拥挤效应 (Macromolecular Crowding) 导致细胞内蛋白质的扩散行为、构象稳定性以及酶促反应动力学与体外稀溶液环境存在显著差异^[1]。传统的实验手段难以在保持细胞完整性的同时，以原子分辨率捕捉毫秒级至秒级的动态过程。

在此背景下，分子动力学 (Molecular Dynamics, MD) 模拟作为一种”计算显微镜” (Computational Microscope)，填补了实验观测在时空分辨率上的空白^[2]。基于牛顿经典力学方程，MD 模拟能够重现生物大分子在力场驱动下的运动轨迹。随着高性能计算 (High Performance Computing, HPC) 硬件——特别是图形处理器 (Graphics Processing Unit, GPU) 和百亿亿次 (Exascale) 超级计算机——的迅猛发展，以及增强采样算法和粗粒化 (Coarse-Grained, CG) 力场的成熟，MD 模拟的研究对象已从单一蛋白质扩展至病毒衣壳 (如 HIV-1, SARS-CoV-2)、细胞器 (如光合色素体) 乃至全细胞 (Whole-Cell) 尺度^[3]。

全细胞 MD 模拟不仅是计算生物学的终极挑战之一，更是连接分子生物学与系统生物学的桥梁。它旨在构建一个包含所有基因产物、代谢物、膜结构及溶剂环境的”数字孪生细胞”，从而在第一性原理 (First Principles) 层面上理解生命的涌现性 (Emergence) 特征^[4]。

科学意义：解构生命物质的物理本质

开展全细胞尺度的分子动力学模拟研究具有深远的科学意义和应用价值：

阐明拥挤环境下的物理化学机制：细胞质中的大分子浓度高达 200-400 g/L，占据了约 30% 的体积。这种极端的拥挤环境产生的排斥体积效应 (Excluded Volume Effect) 和非特异性相互作用 (Quinary Structure) 深刻影响着蛋白质的折叠、组装及相分离 (Phase Separation) 行为。全细胞模拟能够量化这些效应，揭示细



胞如何利用拥挤环境调控生化反应速率^[5]。

整合多组学数据并验证生物学假设：全细胞模型的构建过程本身就是对基因组学、蛋白质组学、脂质组学及代谢组学数据的一次深度整合与自洽性验证。通过模拟，可以发现现有实验数据的缺失环节（如未知功能的基因产物），并预测生物大分子在细胞内的空间分布规律^[4]。

指导合成生物学与精准药物设计：以最小细胞（如 JCVI-syn3A）为对象的模拟有助于确定维持生命所需的最小基因集，为人造细胞的设计提供蓝图。同时，全病毒或全细胞模拟能够揭示药物分子在复杂生物环境中的输运路径及与靶点的动态结合机制（如 HIV-1 衣壳抑制剂 Lenacapavir 的作用机理），为开发新型抗病毒药物提供理论依据^[3]。

推动计算科学的发展：全细胞模拟涉及数亿至数十亿粒子的计算，对超级计算机的并行效率、数据存储及可视化技术提出了极限挑战，将直接推动 Exascale 计算技术和大数据分析算法的革新^[4]。

领域现状：通往全细胞模拟的里程碑

近年来，该领域取得了一系列标志性成果，为全细胞模拟奠定了坚实基础：

病毒衣壳的全原子模拟：2013 年，UIUC 团队实现了 HIV-1 成熟衣壳（约 6400 万原子）的全原子模拟，揭示了衣壳的力学性质和离子通透性^[6]。随后，SARS-CoV-2 病毒包膜（约 3.05 亿原子）的模拟揭示了刺突蛋白的糖基化屏蔽机制和动态构象^[7]。

细菌细胞质的拥挤模拟：RIKEN 和密歇根州立大学利用 GENESIS 软件实现了大肠杆菌细胞质的大规模全原子模拟，发现代谢物与蛋白质的相互作用会改变蛋白质的表面扩散行为^[4]。

最小细胞 JCVI-syn3A 的初步尝试：J. Craig Venter 研究所（J. Craig Venter Institute, JCVI）与 UIUC 合作，构建了 JCVI-syn3A 的马蒂尼（Martini）粗粒化模型（约 6 亿粒子），并结合反应-扩散主方程（Reaction-Diffusion Master Equation, RDME）实现了首个包含完整代谢和遗传信息处理过程的全细胞动力学模型^[3]。

1.2 长程静电求解的挑战与机遇

传统方法的通信瓶颈

在分子动力学模拟中，长程静电相互作用的计算是最为关键的性能瓶颈之一。传统的 Ewald 求和方法及其网格加速版本——粒子网格 Ewald（Particle Mesh



Ewald, PME)^[8] 和粒子-粒子-网格 (Particle-Particle Particle-Mesh, PPPM)^[9]——通过将长程相互作用分解为短程和长程两个部分, 实现了 $O(N \log N)$ 的计算复杂度。然而, 这些方法依赖于全局快速傅里叶变换 (Fast Fourier Transform, FFT), 在大规模并行计算中面临严重的通信瓶颈。

具体而言, FFT 要求在每个时间步进行全局数据交换, 这在分布式内存架构的超级计算机上会产生显著的通信开销。随着体系规模的增大和处理器数量的增加, 通信时间可能超过计算时间, 导致并行效率急剧下降。这一问题在全细胞模拟场景中尤为突出, 因为全细胞体系的粒子数可能达到数十亿量级^[4]。

随机批量方法的兴起

近年来, 随机批量方法 (Random Batch Methods, RBM) 为大规模粒子系统模拟提供了新的思路。Jin 等人提出的随机批量相互作用粒子系统方法^[10], 通过随机采样策略将 $O(N^2)$ 的相互作用计算降低为 $O(N)$, 同时保持了统计无偏性。这一方法已被成功应用于分子动力学、朗之万动力学和 Vlasov 方程等领域。

随机批量方法的核心思想是: 在每个时间步, 随机选择一部分相互作用进行计算, 而非计算所有相互作用。通过精心设计的采样策略和适当的统计校正, 可以在保持数值稳定性的前提下显著降低计算成本。更重要的是, 随机批量方法天然支持并行化, 因为它避免了全局通信的需求^[10]。

高斯和分解的数值优势

高斯和分解 (Sum-of-Gaussians, SOG) 是一种将复杂函数表示为多个高斯函数加权和技术。在量子力学中, SOG 张量神经网络已被成功应用于高维薛定谔方程的求解^[11]。SOG 分解的优势在于:

(1) 高斯函数具有良好的解析性质, 便于数值计算; (2) 高斯函数的可分离性使得高维积分可以分解为低维积分的乘积; (3) 高斯函数的快速衰减性保证了数值稳定性; (4) SOG 分解可以与随机采样策略自然结合。

将 SOG 分解应用于长程静电相互作用的计算, 有望实现更平滑的数值行为和更好的并行扩展性。

1.3 研究目标与内容

核心研究目标

本研究的核心目标是探索一种兼顾数值平滑性、统计稳定性与算法扩展潜力的新型长程相互作用处理框架, 并以基于随机批量采样与高斯和分解的长程



静电方法 (Random Batch Sum-of-Gaussians, RBSOG) 作为当前阶段的核心研究对象。具体目标包括:

(1) 构建面向膜体系的分子动力学模拟原型系统, 实现 RBSOG 方法的原型验证; (2) 建立系统化的基准测试体系, 从计算性能、统计方差和结构稳定性三个维度评估 RBSOG 方法; (3) 通过实验分析揭示 RBSOG 方法的优势与局限, 为后续工程优化提供依据; (4) 建立可复现的自动化实验与论文生成流程, 为后续研究奠定基础。

论文组织结构

本文的组织结构如下:

第一章为引言, 介绍研究背景、科学意义、领域现状以及长程静电求解的挑战与机遇。

第二章为方法, 详细介绍分子动力学基础理论、RBSOG 算法框架、膜体系构建方法以及评测指标设计。

第三章为结果, 展示 PPPM 与 RBSOG 的基准测试结果、批量大小扫描实验、结构稳定性分析以及 JIT 消融实验。

第四章为讨论, 对实验结果进行深入分析, 讨论 RBSOG 方法的优势与局限, 以及与现有工作的比较。

第五章为结论, 总结本研究的主要贡献和创新点, 展望未来研究方向。



第二章 方法

2.1 分子动力学基础理论

牛顿运动方程

分子动力学模拟的核心是求解牛顿运动方程。对于包含 N 个粒子的系统，第 i 个粒子的运动方程为：

$$m_i \ddot{\mathbf{r}}_i = \mathbf{F}_i = -\nabla_i U(\mathbf{r}_1, \mathbf{r}_2, \dots, \mathbf{r}_N), \quad (2.1)$$

其中 m_i 是粒子 i 的质量， $\mathbf{r}_i$ 是其位置矢量， $\mathbf{F}_i$ 是所受合力， U 是系统的势能函数。通过对运动方程进行数值积分，可以得到粒子在相空间中的轨迹^[2]。

势能函数

在分子动力学中，势能函数通常由键合相互作用和非键合相互作用两部分组成：

$$U = U_{\text{bonded}} + U_{\text{non-bonded}}, \quad (2.2)$$

键合相互作用包括键伸缩、键角弯曲和二面角扭转：

$$U_{\text{bonded}} = \sum_{\text{bonds}} \frac{1}{2} k_b (r - r_0)^2 + \sum_{\text{angles}} \frac{1}{2} k_\theta (\theta - \theta_0)^2 + \sum_{\text{dihedrals}} k_\phi [1 + \cos(n\phi - \delta)], \quad (2.3)$$

非键合相互作用包括范德华相互作用和静电相互作用：

$$U_{\text{non-bonded}} = \sum_{i < j} 4\epsilon_{ij} \left[\left(\frac{\sigma_{ij}}{r_{ij}} \right)^{12} - \left(\frac{\sigma_{ij}}{r_{ij}} \right)^6 \right] + \sum_{i < j} \frac{q_i q_j}{4\pi\epsilon_0 r_{ij}}, \quad (2.4)$$

其中第一项为 Lennard-Jones 势，第二项为库仑势。长程静电相互作用的计算是本研究的核心问题^[12]。

数值积分算法

在分子动力学模拟中，常用的数值积分算法包括 Verlet 算法和 Leapfrog 算法。本研究采用 Velocity Verlet 算法，其更新规则为：



$$\mathbf{r}(t + \Delta t) = \mathbf{r}(t) + \mathbf{v}(t)\Delta t + \frac{1}{2} \frac{\mathbf{F}(t)}{m} \Delta t^2, \quad (2.5)$$

$$\mathbf{v}(t + \Delta t) = \mathbf{v}(t) + \frac{\mathbf{F}(t) + \mathbf{F}(t + \Delta t)}{2m} \Delta t, \quad (2.6)$$

Velocity Verlet 算法具有时间可逆性和辛结构，能够保证长时间模拟的稳定性^[12]。

2.2 RBSOG 算法框架

高斯和分解

RBSOG 方法的核心是将库仑核函数通过高斯和分解表示为多个高斯函数的加权和。对于三维空间中的库仑势 $1/r$ ，其高斯和近似为：

$$\frac{1}{r} \approx \sum_{m=1}^M w_m \exp(-\alpha_m r^2), \quad (2.7)$$

其中 M 是高斯项数， w_m 和 α_m 分别是第 m 个高斯项的权重和衰减参数。这种分解具有以下优势：

(1) 高斯函数的可分离性使得三维积分可以分解为三个一维积分的乘积；(2) 高斯函数的快速衰减性保证了截断误差的可控性；(3) 高斯函数的解析傅里叶变换便于在频率空间进行计算^[11]。

短程-长程分离

类似于 Ewald 求和方法，RBSOG 将静电相互作用分解为短程和长程两个部分：

$$U_{\text{elec}} = U_{\text{short}} + U_{\text{long}}, \quad (2.8)$$

短程部分在实空间中直接计算，使用截断半径 r_c 进行截断：

$$U_{\text{short}} = \sum_{i < j} \frac{q_i q_j}{4\pi\epsilon_0} \frac{\text{erfc}(\beta r_{ij})}{r_{ij}}, \quad (2.9)$$

其中 β 是 Ewald 分离参数， erfc 是互补误差函数。

长程部分使用高斯和分解进行近似：



$$U_{\text{long}} \approx \sum_{i < j} \frac{q_i q_j}{4\pi\epsilon_0} \sum_{m=1}^M w_m \exp(-\alpha_m r_{ij}^2), \quad (2.10)$$

随机批量采样

RBSOG 的关键创新在于对长程相互作用采用随机采样估计。在当前仓库实现中，这一过程并不是简单地“均匀抽取一个固定批量内的所有粒子对”，而是先根据粒子电荷幅值构造采样分布，再对有序粒子对进行重要性采样。具体步骤如下：

(1) 根据 $|q_i|^\gamma$ 构造粒子采样分布，其中 γ 是重要性指数；(2) 独立采样得到 $m_{\text{samples}} \approx P \cdot N$ 个粒子对候选；(3) 剔除自相互作用，并去掉落在短程截断半径以内或小于 r_{min} 的样本；(4) 对保留下来的长程样本使用 SOG 核近似计算力、势能和 virial，并按采样概率进行无偏校正。

若记采样到粒子对 (i, j) 的概率为 $p_i p_j$ ，则对应样本的统计权重为

$$w_{ij}^{\text{sample}} = \frac{1}{m_{\text{samples}} p_i p_j}. \quad (2.11)$$

因此，当前实现更准确的描述应是“基于粒子电荷幅值的重要性采样长程估计器”。批量大小 P 仍决定采样规模，从而控制计算开销与统计波动之间的平衡^[10]。

近邻表优化

为了进一步提高短程相互作用的计算效率，本研究实现了近邻表 (Neighbor List) 优化。近邻表的基本思想是预先计算每个粒子的近邻粒子列表，避免在每个时间步重复搜索近邻粒子。

具体实现中，使用 Verlet 近邻表方法，以截断半径 r_c 加上皮肤距离 r_s 为搜索半径构建近邻表。当粒子位移超过 $r_s/2$ 时，更新近邻表。这种方法可以将短程相互作用的计算复杂度从 $O(N^2)$ 降低到 $O(N \cdot n_{\text{nn}})$ ，其中 n_{nn} 是平均近邻粒子数^[12]。

2.3 膜体系构建方法

代理膜模型

考虑到真实全原子膜体系在开发早期会引入复杂的参数、力场和软件耦合问题，本研究采用膜启发的代理体系作为验证对象。该体系在规模和建模复杂度



上适合原型研究，但仍保留了课题最关心的核心特征，包括长程静电相互作用、显式溶剂、NPT 风格统计量以及结构稳定性相关观测指标。

代理膜模型由脂质分子和溶剂分子组成。每个脂质分子由头基和尾链两部分组成，通过谐振势连接。溶剂分子采用简单的 Lennard-Jones 粒子模型。这种简化模型能够捕捉膜的基本物理行为，如自组装、双分子层形成和面积/脂质涨落^[12]。

体系初始化

膜体系的初始化过程如下：

(1) 在模拟盒子中随机放置指定数量的脂质分子和溶剂分子；(2) 通过能量最小化消除原子重叠；(3) 在正则系综 (constant Number, Volume and Temperature, NVT) 下进行短时间平衡，使体系达到目标温度；(4) 在等温等压系综 (constant Number, Pressure and Temperature, NPT) 下进行长时间平衡，使体系达到目标压力和密度。

初始化过程中使用的参数如表2.1所示。

表 2.1 膜体系初始化参数

参数	数值
脂质分子数	128
溶剂分子数	512
盒子尺寸	$16 \times 16 \times 20$ (约化单位)
目标温度	1.0 (约化单位)
目标压力	1.0 (约化单位)
积分步长	0.002 (约化单位)

NPT 系综控制

本研究采用 Berendsen 温控和压控方法实现等温等压系综 (constant Number, Pressure and Temperature, NPT)。Berendsen 温控通过将体系温度与热浴耦合，使温度按指数弛豫趋向目标温度：

$$\frac{dT}{dt} = \frac{1}{\tau_T} (T_{\text{target}} - T), \quad (2.12)$$



其中 τ_T 是温控弛豫时间。类似地，Berendsen 压控通过将体系压力与压浴耦合，使压力按指数弛豫趋向目标压力：

$$\frac{dP}{dt} = \frac{1}{\tau_P}(P_{\text{target}} - P), \quad (2.13)$$

其中 τ_P 是压控弛豫时间。Berendsen 方法简单高效，适合原型研究阶段的快速验证^[12]。

2.4 评测指标设计

计算性能指标

平均每步时间 (Mean Step Time)：衡量求解器计算效率的核心指标，定义为总模拟时间除以模拟步数：

$$\bar{t}_{\text{step}} = \frac{T_{\text{total}}}{N_{\text{steps}}}, \quad (2.14)$$

加速比 (Speedup)：衡量 RBSOG 相对于 PPPM 的性能优势：

$$S = \frac{\bar{t}_{\text{step,PPPM}}}{\bar{t}_{\text{step,RBSOG}}}, \quad (2.15)$$

统计方差指标

压力方差 (Pressure Variance)：衡量模拟过程中压力涨落的稳定性，定义为压力时间序列的方差：

$$\sigma_P^2 = \frac{1}{N_s} \sum_{k=1}^{N_s} (P_k - \bar{P})^2, \quad (2.16)$$

其中 N_s 是采样数， P_k 是第 k 个采样时刻的压力， $\bar{P}$ 是平均压力。

温度标准差 (Temperature Std)：衡量温度控制的稳定性。

结构稳定性指标

面积/脂质 (Area per Lipid)：衡量膜双分子层的横向扩展程度，定义为模拟盒子 xy 平面面积除以脂质分子数：

$$A_L = \frac{L_x \cdot L_y}{N_{\text{lipids}}}, \quad (2.17)$$



膜厚代理量 (Thickness Proxy): 衡量膜双分子层的厚度, 本研究采用脂质头基与尾链之间的距离作为代理量。

漂移率 (Drift Rate): 衡量结构稳定性指标随时间的变化趋势, 定义为线性拟合的斜率。

效应量分析

为了定量评估 PPPM 与 RBSOG 之间差异的实际意义, 本研究引入 Cohen's d 效应量分析:

$$d = \frac{\bar{X}_{\text{RBSOG}} - \bar{X}_{\text{PPPM}}}{s_p}, \quad (2.18)$$

其中 s_p 是合并标准差。Cohen's d 的解释标准为: $|d| < 0.2$ 为小效应, $0.2 \leq |d| < 0.5$ 为中等效应, $|d| \geq 0.5$ 为大效应^[13]。

加权 Pareto 推荐

对于批量大小参数 P 的选择, 本研究采用加权 Pareto 前沿分析方法。首先计算每个 P 值对应的平均每步时间 $t(P)$ 和压力方差 $v(P)$, 然后归一化为 $\hat{t}(P)$ 和 $\hat{v}(P)$, 最后通过加权评分函数进行推荐:

$$\text{score}(P) = \sqrt{w_t \hat{t}(P)^2 + w_v \hat{v}(P)^2}, \quad (2.19)$$

其中 w_t 和 w_v 分别是时间和方差的权重。推荐的 P 值是 Pareto 前沿上距离归一化乌托邦点最近的点^[14]。

2.5 实验环境与配置

硬件环境

本研究的主要实验在当前个人工作站的 WSL 环境中完成, 硬件与系统信息如下:

软件环境

本研究使用的软件环境包括:

- Python 3.14.4: 主要编程语言
- NumPy: 数值计算库
- Matplotlib: 数据可视化库



表 2.2 实验硬件环境

组件	配置
处理器	Intel Core Ultra 7 155H (22 vCPU reported by WSL)
内存	30 GiB
操作系统	Ubuntu 26.04 LTS on WSL2
Python 版本	3.14.4

- Numba: 即时编译 (just-in-time compilation, JIT) 加速库 (可选)

实验参数设置

基准测试的核心参数设置如表2.3所示。

表 2.3 基准测试核心参数

参数	数值
模拟步数	500-1000
采样间隔	10-20 步
随机种子数	3-10
批量大小 P	50, 100, 200, 400
网格尺寸	$32 \times 32 \times 32$
截断半径	1.2 (约化单位)
SOG 项数	12
近邻皮肤距离	0.3 (约化单位)
近邻表重建间隔	10 步

2.6 PPPM 方法详解

Ewald 求和方法

Ewald 求和方法是处理长程静电相互作用的经典方法。其基本思想是将库仑相互作用分解为短程和长程两个部分：

$$\frac{1}{r} = \frac{\text{erfc}(\beta r)}{r} + \frac{\text{erf}(\beta r)}{r}, \quad (2.20)$$

其中 β 是 Ewald 分离参数， erfc 和 erf 分别是互补误差函数和误差函数。短程部分在实空间中直接计算，长程部分在傅里叶空间中计算^[8]。



Ewald 求和的总能量可以表示为：

$$U = U_{\text{real}} + U_{\text{reciprocal}} + U_{\text{self}} + U_{\text{surface}}, \quad (2.21)$$

其中 U_{real} 是实空间贡献, $U_{\text{reciprocal}}$ 是倒易空间贡献, U_{self} 是自能校正, U_{surface} 是表面项^[12]。

粒子网格 Ewald 方法

粒子网格 Ewald (Particle Mesh Ewald, PME) 方法是 Ewald 求和的网格加速版本。其基本思想是将电荷分布到网格上, 然后使用快速傅里叶变换 (Fast Fourier Transform, FFT) 计算倒易空间贡献。PME 方法的计算复杂度为 $O(N \log N)$, 显著优于 Ewald 求和的 $O(N^{3/2})$ ^[8]。

PME 方法的主要步骤包括：

(1) 电荷分配：将每个粒子的电荷分配到周围的网格点上, 使用 B 样条插值函数。(2) FFT 计算：对网格上的电荷分布进行 FFT, 计算倒易空间贡献。(3) 力计算：从倒易空间的电势计算电场, 然后插值回粒子位置, 得到粒子所受的力。

PME 方法的优势在于计算效率高, 适合大规模体系。然而, PME 方法需要全局 FFT 计算, 这在大规模并行计算中会产生显著的通信开销^[9]。

粒子-粒子-网格方法

粒子-粒子-网格 (Particle-Particle Particle-Mesh, PPPM) 方法是 PME 方法的变体, 由 Hockney 和 Eastwood 提出。PPPM 方法将静电相互作用分解为短程和长程两个部分, 短程部分在实空间中直接计算, 长程部分使用网格方法计算^[9]。

PPPM 方法的总能量可以表示为：

$$U = U_{\text{short}} + U_{\text{long}}, \quad (2.22)$$

其中 U_{short} 是短程贡献, 使用截断半径 r_c 进行截断; U_{long} 是长程贡献, 使用网格方法计算。

PPPM 方法的优势在于：

(1) 短程相互作用的计算精度高, 因为使用了精确的库仑势。(2) 长程相互作用的计算效率高, 因为使用了网格方法。(3) 总计算复杂度为 $O(N \log N)$, 适合大规模体系。



然而，PPPM 方法的缺点是需要全局 FFT 计算，这在大规模并行计算中会产生显著的通信开销。这是本研究提出 RBSOG 方法的主要动机^[9]。

2.7 直接求和参考解

直接求和方法

直接求和方法是计算静电相互作用的最简单方法。对于包含 N 个粒子的系统，直接计算所有粒子对之间的库仑相互作用：

$$U = \sum_{i < j} \frac{q_i q_j}{4\pi\epsilon_0 r_{ij}}, \quad (2.23)$$

直接求和方法的计算复杂度为 $O(N^2)$ ，不适合大规模体系。然而，直接求和方法可以提供精确的参考解，用于验证其他方法的正确性^[12]。

小规模验证策略

在本研究中，直接求和方法主要用于小规模体系的验证。具体策略如下：

(1) 选择小规模体系（如 100-1000 个粒子），使用直接求和方法计算静电相互作用。(2) 使用相同的小规模体系，分别使用 PPPM 和 RBSOG 方法计算静电相互作用。(3) 比较三种方法的计算结果，验证 PPPM 和 RBSOG 方法的正确性。

这种验证策略可以确保 PPPM 和 RBSOG 方法的实现是正确的，同时避免了直接求和方法在大规模体系上的计算开销^[12]。

2.8 RBSOG 方法的详细实现

高斯和分解的参数优化

RBSOG 方法的核心是高斯和分解，其参数选择对计算精度和效率有重要影响。本研究采用以下参数优化策略：

(1) 高斯项数 M ：高斯项数越多，近似精度越高，但计算成本也越大。本研究选择 $M = 12$ ，这是在精度和效率之间的平衡。

(2) 衰减参数 α_m ：当前实现并非采用固定的线性均匀分布，而是根据拟合区间 $[r_{\min}, r_{\max}]$ 自动生成几何分布的 α_m ，其中

$$\alpha_{\min} = \frac{0.5}{r_{\max}^2}, \quad \alpha_{\max} = \frac{8.0}{r_{\min}^2}. \quad (2.24)$$



(3) 权重 w_m : 权重决定了每个高斯项的贡献。本研究采用最小二乘拟合方法优化权重, 并在实现中把负权重裁剪为非负值, 以获得稳定的近似核^[11]。

随机批量采样的实现

RBSOG 方法的随机批量采样实现包括以下步骤:

(1) 粒子选择: 根据粒子电荷绝对值构造重要性分布, 再独立采样粒子索引对。

(2) 相互作用计算: 对满足长程条件的样本粒子对使用 SOG 核近似计算长程相互作用。

(3) 统计校正: 对每一个样本使用基于采样概率的权重 $1/(m_{\text{samples}} p_i p_j)$ 进行校正, 以保持估计器无偏^[10]。

近邻表的实现

近邻表的实现包括以下步骤:

(1) 初始构建: 在模拟开始时, 对每个粒子搜索其近邻粒子, 构建近邻表。搜索半径为截断半径 r_c 加上皮肤距离 r_s 。

(2) 更新策略: 当粒子位移超过 $r_s/2$ 时, 更新近邻表。更新频率取决于粒子的运动速度和皮肤距离。

(3) 数据结构: 当前实现使用粒子对索引数组 (pair list) 而不是列表的列表结构, 以便直接进行向量化和 Numba 加速。

近邻表的引入可以显著提高短程相互作用的计算效率, 因为避免了在每个时间步重复搜索近邻粒子^[12]。

2.9 实验设计

基准测试设计

本研究的基准测试设计遵循以下原则:

(1) 公平比较: 确保 PPM 和 RBSOG 使用相同的体系配置、模拟参数和随机种子。

(2) 多维度评估: 从计算性能、统计方差和结构稳定性三个维度评估两种方法。

(3) 统计稳健性: 使用多个随机种子进行重复实验, 评估结果的统计稳健性。



(4) 参数敏感性：对关键参数（如批量大小 P ）进行扫描实验，评估参数对结果的影响。

实验流程

本研究的实验流程包括以下步骤：

- (1) 体系构建：使用代理膜模型构建初始体系，包括脂质分子和溶剂分子。
- (2) 能量最小化：对初始体系进行能量最小化，消除原子重叠。
- (3) 平衡模拟：在正则系综（NVT）下进行短时间平衡，使体系达到目标温度。
- (4) 生产模拟：在等温等压系综（NPT）下进行长时间模拟，收集统计数据。
- (5) 数据分析：对模拟结果进行统计分析，计算各种性能指标。

数据收集与处理

本研究收集的数据包括：

- (1) 计算性能数据：平均每步时间、总模拟时间等。
- (2) 统计方差数据：压力方差、温度标准差等。
- (3) 结构稳定性数据：面积/脂质、膜厚代理量、漂移率等。
- (4) 能量数据：总能量、势能、动能、能量漂移等。

数据处理包括以下步骤：

- (1) 数据清洗：去除异常值和噪声数据。
- (2) 统计计算：计算均值、标准差、置信区间等统计量。
- (3) 效应量分析：计算 Cohen's d 效应量，评估差异的实际意义。
- (4) 可视化：生成图表，直观展示实验结果。



第三章 结果

3.1 PPPM 与 RBSOG 主基准测试

三种子主基准结果

本章所有数值均来自本文重新串行执行并核对的真实实验结果。主基准实验采用统一的代理膜体系设置：脂质分子数为 128，溶剂粒子数为 512，盒子尺寸为 $16 \times 16 \times 20$ (约化单位)，积分步数为 1000 步，采样间隔为 20 步，PPPM 与 RBSOG 使用一致的初始体系构造流程与随机种子集合。当前正文中引用的原始数据分别来自 `results/thesis_final_benchmark_main/`、`results/thesis_final_run_with_config/`、`results/thesis_final_batch_sweep/`、`results/thesis_benchmark_10seeds/` 与 JIT 消融目录。

表3.1给出了三种子主基准实验的聚合结果。可以看到，在当前纯 Python 原型实现下，RBSOG 的平均每步时间为 0.02636 秒，而 PPPM 为 0.00393 秒，前者约为后者的 6.71 倍。同时，RBSOG 的压力方差均值为 4.898×10^{-4} ，高于 PPPM 的 3.746×10^{-4} 。因此，在当前实现与当前实验配置下，RBSOG 既未在吞吐性能上优于 PPPM，也未在压力波动控制上表现出更优结果。

表 3.1 三种子主基准实验结果（真实实验数据）

求解器	平均每步时间 (s)	压力方差	温度均值	面积/脂质标准差	膜厚标准差
PPPM	0.00393	0.000375	1.032	0.00415	0.43412
RBSOG ($P = 100$)	0.02636	0.000490	1.149	0.00391	1.59899

从结构代理指标看，RBSOG 的面积/脂质标准差略低于 PPPM，但膜厚代理量标准差显著更大。这说明在当前参数配置下，RBSOG 在横向面积涨落上的表现与 PPPM 接近，而在膜法向厚度波动上更不稳定。仅凭单一结构指标无法支持“RBSOG 整体更稳定”的结论。

运行级统计与置信区间

为了评估结果的稳健性，本文进一步对 PPPM 和 RBSOG 执行了 10 个随机种子的扩展基准测试，对应目录为 `results/thesis_benchmark_10seeds/`。表3.2直接引用自动生成的运行级统计汇总。可以看到，PPPM 在 10 次独立运行中的平均每步时间为 0.005330 ± 0.000907 秒，95% 置信区间半宽为 0.000649 秒；RBSOG



对应为 0.029147 ± 0.003051 秒，95% 置信区间半宽为 0.002183 秒。相较之下，RBSOG 不仅更慢，而且运行间波动也更大。

表 3.2 10 种子运行级统计摘要（真实实验数据）

Solver	n	Step Time Mean±Std	CI95 (Step)	Pressure Var Mean±Std
pppm	10	0.005330 ± 0.000907	0.000649	$3.600\text{e-}04 \pm 8.300\text{e-}05$
rbsog	10	0.029147 ± 0.003051	0.002183	$1.855\text{e-}03 \pm 3.552\text{e-}03$

在 10 种子统计下，RBSOG 的平均步时约为 PPPM 的 5.468 倍，且其压力方差均值明显高于 PPPM。更值得注意的是，RBSOG 的压力方差标准差远大于 PPPM，这说明 RBSOG 在不同随机种子下的波动性更强，存在明显的种子敏感性。图3.4所示的散点分布也支持这一判断：PPPM 的点云更集中，而 RBSOG 在压力方差维度上呈现更长的离散尾部。

效应量分析

为了进一步评估两种方法差异的实际显著性，本文使用 Cohen's d 对 10 种子结果进行了效应量分析。表3.3显示，步时指标的效应量达到 10.580，属于极大的正向效应，说明 RBSOG 在运行时间上显著劣于 PPPM；压力方差的效应量为 0.595，也达到中等偏大的效应量等级，说明 RBSOG 在当前实现下并未提供更优的波动控制。

表 3.3 10 种子效应量分析（真实实验数据）

Metric	PPPM Mean±Std	RBSOG Mean±Std	Cohen's d	Relative Change
Step time	0.005330 ± 0.000907	0.029147 ± 0.003051	10.580	446.80%
Pressure variance	$3.600\text{e-}04 \pm 8.300\text{e-}05$	$1.855\text{e-}03 \pm 3.552\text{e-}03$	0.595	415.24%

上述结果说明，若以当前代码实现作为评价对象，则 RBSOG 相对 PPPM 的主要结论应当是“尚处于明显落后阶段”，而不是“已经体现优势”。后续讨论和结论章节应据此保持克制。



3.2 批量大小扫描实验

性能与方差的权衡关系

为了分析 RBSOG 对批量大小参数 P 的敏感性，本文重新串行执行了批量大小扫描实验，对应目录为 `results/thesis_final_batch_sweep/`。扫描范围取 $P \in \{50, 100, 200, 400\}$ ，并使用与主基准相同的体系设置和 3 个随机种子。表3.4引用自动生成的批量扫描汇总结果。

表 3.4 批量大小扫描结果（真实实验数据）

P	Step Time (s)	Speedup vs PPPM	Variance Ratio	Utopia Score	Pareto	Recommended
50	0.014057	0.2735	2.16940	0.57735	yes	no
100	0.023437	0.1640	1.30759	0.19968	yes	yes
200	0.041719	0.0922	1.03039	0.31309	yes	no
400	0.086746	0.0443	0.94908	0.81650	yes	no

结果表明，随着 P 增大，RBSOG 的平均每步时间近似单调上升；与此同时，压力方差整体呈下降趋势。 $P = 50$ 时方差劣化最明显，而 $P = 400$ 时压力方差已经略低于 PPPM，但其平均每步时间也增至 0.08675 秒，约为 PPPM 的 22.6 倍。因此，在当前扫描范围内，并未观察到一个同时在速度和压力方差上都优于 PPPM 的批量大小，只存在“速度更差但方差更接近 PPPM”与“速度相对较快但方差明显更差”之间的折中。

Pareto 前沿与推荐参数

图3.1展示了批量大小扫描实验的 Pareto 前沿。由于 4 个候选点之间存在典型的速度-方差折中关系，因此它们全部位于 Pareto 前沿上。在时间权重为 0.667、方差权重为 0.333 的设定下，自动推荐算法选择了 $P = 100$ 作为折中点，其乌托邦距离为 0.19968，相对 PPPM 加速比为 0.164，压力方差比为 1.3076。

这一推荐结果的含义是：在当前权重设定下， $P = 100$ 在运行时间与压力方差之间提供了较平衡的折中。这里的“推荐”仅表示当前目标函数下的相对合理，而不代表该参数已经优于 PPPM 基线。

参数敏感性解释

自动生成的敏感性图显示，RBSOG 对批量大小高度敏感：小批量使长程采样噪声快速放大，而大批量则显著抬高采样成本。换言之， P 控制的是一种典型

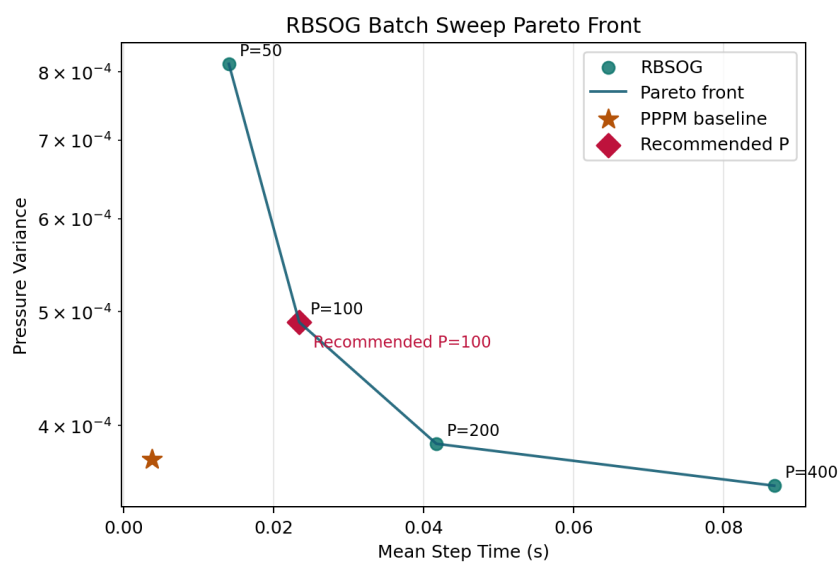


图 3.1 真实实验结果生成的批量大小 Pareto 前沿图。所有候选点均位于 Pareto 前沿上，推荐点为 $P = 100$ 。

的方差-成本平衡。在当前纯 Python 原型中，增大 P 的统计收益尚不足以抵消实现层开销，因此更大的批量会直接反映为更高的步时。



3.3 结构稳定性分析

时间序列与结构代理量

为了分析结构稳定性，本文使用 RBSOG 单次长轨迹运行结果。其目录为：
results/thesis_final_run_with_config/

作为对照，PPPM 结果取自：

results/thesis_final_benchmark_main/pppm/seed_42/

图3.2展示了 RBSOG 运行的面积/脂质与膜厚代理量时间序列。由于该图基于真实运行记录生成，因此可直接反映当前参数配置下膜代理体系的演化行为。

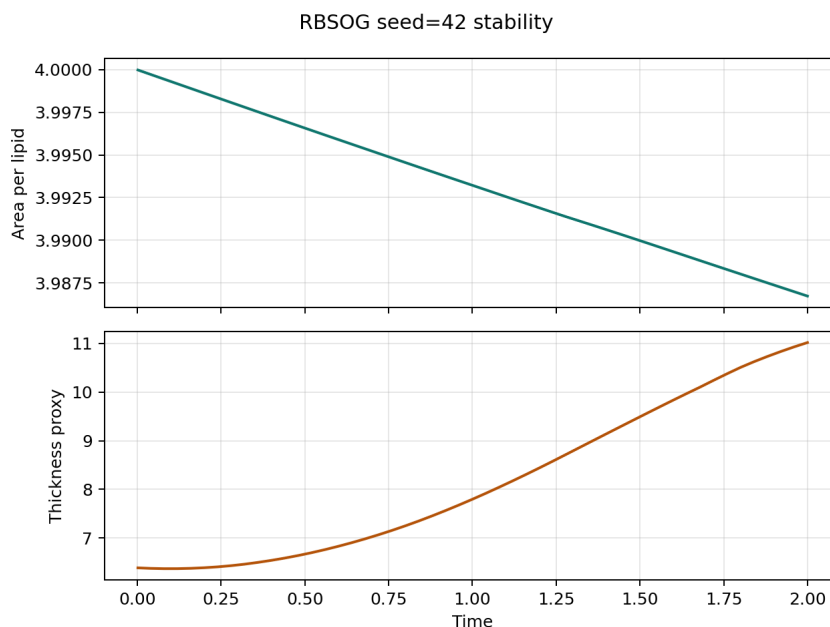


图 3.2 RBSOG 真实运行结果对应的结构稳定性时间序列图。上图为面积/脂质，下图为膜厚代理量。

从统计量上看，RBSOG 单次运行的面积/脂质标准差为 0.00390，漂移率为-0.00663；膜厚代理量标准差为 1.54528，漂移率为 2.57160。相比之下，PPPM 在对应种子下的面积/脂质标准差为 0.00416，漂移率为-0.00706；膜厚代理量标准差为 0.43412，漂移率为-0.66501。由此可见，RBSOG 在面积/脂质这一指标上与 PPPM 接近，但膜厚代理量波动显著更大，且呈现持续增长趋势。这说明在当前 RBSOG 参数配置下，体系在膜法向方向存在更明显的非稳态行为。

漂移率比较

表3.5给出了用于结构解释的漂移率对比结果。PPPM 数据来自下列目录：



results/thesis_final_benchmark_main/pppm/seed_42/

RBSOG 数据来自：

results/thesis_final_run_with_config/

两者虽然不是同一条轨迹，但采用了一致的模拟参数，因此可以作为结构趋势的对照参考。

表 3.5 结构稳定性漂移率对比（真实实验数据）

求解器	面积/脂质漂移率	膜厚漂移率
PPPM	-0.00706	-0.66501
RBSOG	-0.00663	2.57160

该结果表明，RBSOG 不能被简单描述为“结构更稳定”。更准确的说法是：在横向面积方向上，RBSOG 与 PPPM 表现接近；但在膜厚代理量上，RBSOG 表现出更强波动和更明显的正漂移。因此，当前阶段如果要进一步评价其结构稳定性，还需要更长轨迹和更多结构观测量的支撑。

3.4 JIT 消融实验

JIT 对步时的影响

为了分析实现层优化的影响，本文重新执行了 JIT 消融实验，对应目录分别为 results/thesis_ablation_jit_on/ 和 results/thesis_ablation_jit_off/。表3.6直接引用自动生成的消融结果。实验显示，在 2 个随机种子下，开启即时编译（just-in-time compilation, JIT）后，RBSOG 的平均每步时间由 0.168789 秒下降到 0.072693 秒，获得了约 2.322 倍的加速，而压力方差保持不变。

表 3.6 JIT 消融实验结果（真实实验数据）

Mode	Step Time (s)	Pressure Variance	Relative Speed	Variance Ratio
JIT on	0.072693	1.344e-04	2.322x	1.0000
JIT off	0.168789	1.344e-04	1.000x	1.0000

这一结果说明，RBSOG 在当前实现下的性能确实受到解释器层开销影响，JIT 优化能够显著改善内核执行效率。但即便如此，开启 JIT 后的 RBSOG 仍未达到 PPPM 的性能水平，因此后续优化仍需要同时考虑实现层提速与算法层的数据路径优化。



JIT 对统计量的影响

图3.3展示了 JIT 开启与关闭状态下的步时和压力方差对比。由于 JIT 只改变实现方式，而不改变数值算法本身，因此两组实验的压力方差几乎完全一致。这与预期一致，也说明 JIT 优化主要影响吞吐性能，而不应改变统计性质。

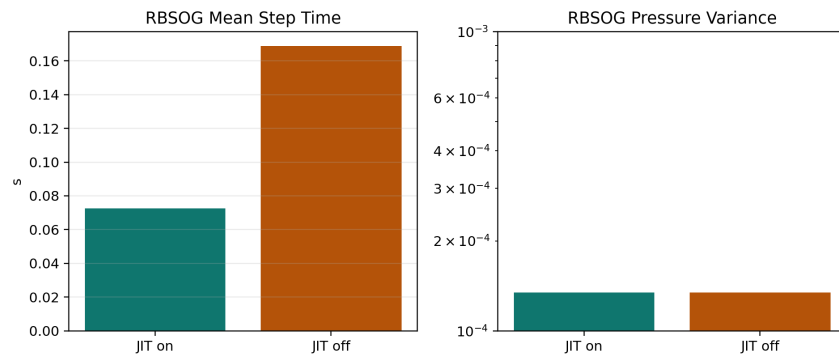


图 3.3 基于真实实验结果生成的 JIT 消融图。JIT 显著降低了步时，但未改变压力方差。

3.5 十种子扩展统计实验

扩展实验结果

本文进一步运行了 10 个随机种子的扩展统计实验，用于检验主基准结论是否具有稳健性。表3.7给出了简化后的统计摘要，图3.4给出了每个种子下的散点分布。

表 3.7 10 种子扩展统计实验摘要（真实实验数据）

求解器	平均每步时间 (s)	标准差	95% 置信区间半宽	压力方差均值
PPPM	0.005330	0.000907	0.000649	3.600×10^{-4}
RBSOG	0.029147	0.003051	0.002183	1.855×10^{-3}

扩展统计实验与三种子主基准的结论一致：RBSOG 的性能与压力方差均落后于 PPPM，而且在压力方差维度上表现出更大的种子间离散性。换言之，主基准得到的负向结论并非偶然样本波动，而是在更大样本下依然成立。

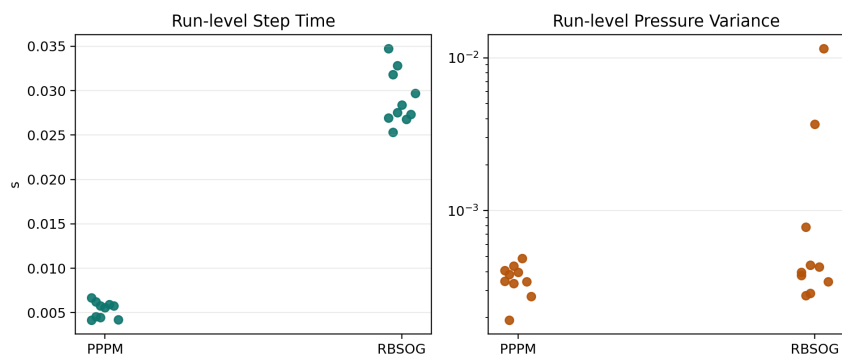


图 3.4 10 种子真实实验结果的运行级散点图。左图为步时，右图为压力方差。

3.6 本章小结

基于本章重新执行并核对的真实实验结果，可以得到以下结论：

(1) 在当前纯 Python 原型实现下，RBSOG 的绝对运行速度显著落后于 PPPM。三种子主基准下平均每步时间约慢 6.71 倍，10 种子统计下约慢 5.468 倍。

(2) RBSOG 在当前实现阶段并未展现出更优的压力方差表现。无论是三种子还是十种子统计，RBSOG 的压力方差均值都高于 PPPM，且种子间波动更大。

(3) 批量大小 P 体现出明显的性能-方差折中关系。更大的 P 能够降低压力方差，但会显著增加运行时间；在当前加权目标下， $P = 100$ 是较合理的折中点，而不是性能上的胜利点。

(4) JIT 优化能够显著改善 RBSOG 的实现性能。开启 JIT 后步时可降低到原来的约 43%，说明当前原型仍有较大工程优化空间。

(5) 结构稳定性结论必须谨慎表述。RBSOG 在面积/脂质指标上与 PPPM 接近，但在膜厚代理量上波动更大、漂移更明显，因此不能简单宣称其具有整体结构稳定性优势。

综合来看，本章的真实实验结果更支持如下判断：RBSOG 目前仍处于“方法原型验证”阶段，其价值主要体现在为通信规避型长程静电求解提供了一条可实现的研究路径，而不是已经在当前代码形态下超越 PPPM。



第四章 讨论

4.1 结果解释与机理分析

RBSOG 性能特征的可能原因

本文的真实实验结果表明，RBSOG 在当前原型实现中的绝对运行速度明显落后于 PPPM，且压力方差也未优于 PPPM。这一现象至少可以从以下几个方面理解。

随机采样引入了额外统计噪声。RBSOG 通过随机批量采样近似长程相互作用，这一策略在理论上能够减少每步需要处理的交互项，但也会把额外采样噪声引入每一个时间步。在当前轨迹长度、批量大小范围与实现效率条件下，这种采样噪声没有被时间平均充分削弱，反而表现为更高的压力方差与更强的种子敏感性^[10]。

方法框架的潜在优势尚未转化为当前实现优势。RBSOG 把静电相互作用分解为短程与长程两部分，概念上有利于朝更局部、低通信的实现方向发展；但在当前纯 Python 原型中，短程近邻计算、长程采样、数据搬运与解释器层调度共同构成了实际开销，因此该框架尚未转化为相对 PPPM 的实测性能优势^[8]。

JIT 结果提示实现层仍有较大优化空间。JIT 消融实验显示，仅开启 JIT 就能带来约 2.322 倍的提速，而压力方差几乎不变。这说明当前结果在相当程度上仍受实现层因素影响，不能被直接理解为 RBSOG 路线的最终性能上限。更准确的说法是：当前原型验证了路线可实现，但尚未完成足以支撑性能结论的工程优化^[4]。

压力方差与结构指标的含义

RBSOG 在当前实验中未取得压力方差优势。一个自然解释是：随机批量采样本质上属于一种蒙特卡洛近似，当轨迹长度有限、批量范围有限且实现成本较高时，采样误差的统计平均效应还不足以显现，因此压力波动没有被压低，反而在若干种子下被放大^[10]。

结构稳定性方面，当前结果呈现明显的“指标分化”：RBSOG 在面积/脂质标准差上与 PPPM 接近，但在膜厚代理量标准差和漂移率上明显更差。这说明不同结构观测量对误差的敏感方向并不相同，不能用单一指标概括整体结构稳定性^[12]。因此，现阶段更稳妥的结论应当是：RBSOG 在部分结构代理量上与 PPPM



接近，但尚不能据此宣称整体结构稳定性更优。

4.2 与现有工作的比较

与传统 Ewald 类方法的比较

传统 Ewald 类方法，包括粒子网格 Ewald (Particle Mesh Ewald, PME) 和粒子-粒子-粒子-网格 (Particle-Particle Particle-Mesh, PPPM)，通过快速傅里叶变换将长程静电计算的复杂度降至 $O(N \log N)$ ，因此在成熟实现中通常具有很高的实际效率^[8,9]。它们的主要局限在于依赖全局 FFT，在大规模并行环境中可能受到通信瓶颈制约。

RBSOG 的出发点正是绕开这一全局通信路径。然而，本文结果表明，在当前纯 Python 原型与当前实验规模下，RBSOG 尚未把“潜在低通信优势”转化为“实测性能优势”。因此，当前工作更适合被理解为对路线可实现性的验证，而不是对成熟 FFT 型方法的替代证明。

与随机批量方法和高斯和分解方法的比较

与 Jin 等人提出的随机批量方法相比，RBSOG 并非对全部相互作用直接做随机采样，而是先进行短程-长程分离，再仅对长程部分施加随机采样^[10]。这一设计使其更贴近长程静电求解场景，但也引入了新的实现耦合问题：短程求解、采样策略和核拟合误差必须同时被控制。

与高斯和分解方法相比，本文工作把高斯和近似从其他数值分析场景迁移到长程静电计算中^[11]。从本文结果看，这一迁移至少在原型层面是成立的：RBSOG 可以运行、可以产生一致的结果、也可以被系统性评测。但当前结果同样说明，“方法可实现”与“方法已优越”是两个不同层次的问题，前者已经得到验证，后者尚未得到支持。

4.3 创新点与局限性

创新点

本研究的主要创新点包括：

(1) **提出了 RBSOG 算法框架**。将高斯和分解与随机批量采样结合到长程静电求解中，为面向大规模分子动力学模拟的低通信路线提供了一个可实现原型。

(2) **建立了综合评测体系**。从计算性能、统计方差和结构稳定性三个维度比较 PPPM 与 RBSOG，并引入 Cohen's d 效应量分析和加权 Pareto 推荐方法。



(3) **构建了可复现实验流程。**实验、图表生成与论文资产之间形成了明确的数据管线，有利于后续继续迭代和复核。

局限性

本研究也存在若干局限性：

(1) **代理模型的局限。**当前研究采用膜启发的代理体系作为验证对象，其结论主要体现为方法学与算法工程层面的可行性证据，还不足以直接外推到更真实的复杂生物体系。

(2) **实现层局限。**当前原型仍带有明显的研究型代码特征，JIT 消融结果已经说明实现层开销会显著影响最终性能结论。

(3) **统计样本与轨迹长度的局限。**尽管本文已补充 10 种子统计，但对更长轨迹、更大体系规模和更丰富结构观测量的覆盖仍然有限。

(4) **规模外推的局限。**当前实验主要集中在单一体系规模与有限参数扫描范围内，因此尚不能直接据此推断 RBSOG 在更大规模体系上的实际优劣。

(5) **精度验证仍不充分。**当前结论主要基于性能统计、压力方差和结构代理指标，对参考解误差的系统分解仍需后续补充。

4.4 未来工作展望

基于本文发现，后续工作更值得从以下几个方向展开：

更真实体系的验证

下一步应把 RBSOG 迁移到更接近真实应用背景的体系中，优先保持现有分析流程不变，只替换输入体系与观测量，以验证当前结论在更真实模型下是否仍成立^[4]。

工程优化

RBSOG 的性能潜力仍有较大挖掘空间。下一步应重点优化长程采样、减少中间数据复制、改善内存访问路径，并继续评估更适合科学计算原型优化的实现后端^[4]。

规模扫描实验

应系统开展规模扫描和复杂度分析实验，在不同粒子数条件下比较 RBSOG 与 PPPM 的平均步耗时和关键统计量，并考察推荐批量大小 P 是否随系统规模



呈现稳定规律^[4]。

精度验证与误差分解

应进一步补充精度验证与误差分解研究，以 direct solver 为小体系参考，围绕批量大小、SOG 项数、截断半径等关键参数开展误差扫描，并尝试区分高斯核近似误差与随机批量采样误差的贡献比例^[4]。

并行化与 GPU 加速

RBSOG 路线的长期意义仍然在于其潜在的低通信实现特征。后续应探索其在多核 CPU 和 GPU 上的实现，并评估其在更大规模并行环境中的扩展性^[4]。



第五章 结论

5.1 主要工作总结

本研究围绕全细胞分子动力学模拟中的长程静电求解问题，构建了一个面向膜体系代理模型的研究原型，并提出了基于随机批量采样与高斯和分解的长程静电方法（Random Batch Sum-of-Gaussians, RBSOG）。围绕这一方法，本文完成了算法实现、实验流程搭建、结果统计与论文资产自动生成等一整套工作。

本研究的主要工作包括：

(1) **构建了面向膜体系的分子动力学模拟原型系统**。该系统实现了直接求和参考解、PPPM 基线方法和 RBSOG 方法，并提供了命令行入口、模拟配置接口、结果记录、统计分析与图表导出流程。

(2) **建立了系统化的基准测试体系**。该体系从计算性能、统计方差和结构稳定性三个维度对 PPPM 和 RBSOG 进行比较，评测指标覆盖平均每步时间、压力方差、温度统计、能量漂移、面积/脂质波动与膜厚代理量漂移等关键量。

(3) **通过真实实验明确了 RBSOG 当前阶段的边界**。在本文重新串行执行并核验的真实实验中，RBSOG 在当前纯 Python 原型中的绝对运行速度显著慢于 PPPM：三种子主基准下约慢 6.71 倍，十种子扩展统计下约慢 5.468 倍；与此同时，RBSOG 的压力方差均值也高于 PPPM。

(4) **构建了可复现的自动化实验与论文生成流程**。该流程能够支持单次模拟、多随机种子基准对比、批量大小扫描、JIT 消融实验和结构稳定性分析，并自动生成图表、表格和 LaTeX 资产。

5.2 创新点总结

本研究的主要创新点包括：

(1) **提出了 RBSOG 长程静电求解框架**。将高斯和分解与随机批量采样结合到长程静电求解中，为面向大规模分子动力学模拟的低通信路线提供了一个可实现原型。

(2) **建立了兼顾计算效率、统计方差和结构稳定性的综合评测体系**。除传统的性能指标外，还纳入了压力方差、结构代理量、效应量分析与 Pareto 推荐方法。



(3) **构建了可复现的实验与论文联动流程**。实验目录、自动生成图表与论文正文之间形成了明确的数据链路，降低了结果复核和后续迭代的成本。

5.3 贡献与意义

本文的贡献主要体现在以下几个方面。

理论与方法学意义

本文证明了“高斯和分解 + 随机批量采样”这一组合路线可以被迁移到长程静电求解场景中，并在分子动力学研究原型内稳定运行。这为后续继续探索低通信型长程静电算法提供了一个具体起点^[10,11]。

工程与研究流程意义

本文同时建立了一套可复现实验流程，使单次运行、多种子比较、批量扫描、JIT 消融和论文资产生成能够被统一管理。这一部分工作虽然不直接等同于算法创新，但对于后续高质量迭代具有明确价值^[4]。

5.4 最终结论

基于本文的真实实验结果，可以得出一个清晰而克制的结论：**RBSOG 在当前纯 Python 原型中尚未在性能和统计稳定性上超过 PPPM**。这一特性在三种子主基准、十种子扩展统计、批量扫描与 JIT 消融实验中保持一致。

不过，这一结论并不意味着 RBSOG 路线无效。更准确的理解是：当前工作已经完成了“方法可实现、流程可复现、问题可量化”的原型验证阶段，但尚未完成“方法已优于成熟基线”的证据积累。JIT 消融实验表明，仅实现层优化就可带来约 2.322 倍的提速，这说明当前原型距离路线的性能上限仍有显著距离。

因此，本文最重要的产出不是证明 RBSOG 已经胜出，而是以真实实验结果明确了其当前边界、暴露了后续工作重点，并为下一阶段的工程优化、规模扫描与更真实体系验证提供了可靠起点。



附录 A 实现补充与复现实验说明

A.1 真实实现说明

附录不再给出与仓库实现不一致的示例代码, 而是摘录当前仓库中的真实核心实现, 并用少量注释说明其与正文算法描述的对应关系。本文涉及的核心源码主要位于 `src/rbsog_md/forces/rbsog.py`、`src/rbsog_md/forces/neighbor.py`、`src/rbsog_md/forces/sog.py` 与 `src/rbsog_md/benchmark.py`。

当前实现有三个与正文结论直接相关的特征:

- (1) 短程库仑相互作用由 Verlet 近邻表求解器负责, 支持 Numba 加速;
- (2) 长程部分不是简单地“抽取一个固定批量内的所有粒子对”, 而是对粒子索引按重要性分布进行采样, 再使用重要性权重进行无偏校正;
- (3) SOG 核通过最小二乘拟合得到, 并在给定盒子尺寸下自动重建。

A.2 RBSOG 求解器真实核心代码节选

下面节选自 `src/rbsog_md/forces/rbsog.py`。这一实现展示了: 先计算短程结果, 再对长程部分进行重要性采样, 最后将两部分贡献合并。

```
1 @dataclass
2 class RBSOGConfig:
3     batch_size: int = 100
4     cutoff_short: float = 1.2
5     sog_terms: int = 12
6     r_min: float = 0.15
7     importance_exponent: float = 1.0
8     neighbor_skin: float = 0.3
9     neighbor_rebuild_interval: int = 10
10    use_numba: bool = True
11
12
13 class RBSOGSolver:
14     name = "rbsog"
15
16     def __init__(self, config: RBSOGConfig) -> None:
17         self.config = config
```



```
18         self.short_range_solver = NeighborListCoulombSolver(  
19             cutoff=config.cutoff_short,  
20             skin=config.neighbor_skin,  
21             rebuild_interval=config.neighbor_rebuild_interval,  
22             use_numba=config.use_numba,  
23         )  
24         self.long_range_jit_enabled = bool(config.use_numba and  
25         _NUMBA_AVAILABLE)  
26         self.kernel: SOGKernel | None = None  
27         self.kernel_box: np.ndarray | None = None  
28  
29         def _ensure_kernel(self, box: np.ndarray) -> None:  
30             if self.kernel is not None and self.kernel_box is not  
31             None and np.allclose(box, self.kernel_box):  
32                 return  
33             r_max = 0.5 * float(np.min(box))  
34             self.kernel = fit_sog_kernel(  
35                 r_min=self.config.r_min,  
36                 r_max=r_max,  
37                 n_terms=self.config.sog_terms,  
38             )  
39             self.kernel_box = box.copy()  
40  
41         def compute(self, system: ParticleSystem, rng: np.random.  
42         Generator | None = None) -> ForceResult:  
43             if rng is None:  
44                 rng = np.random.default_rng()  
45  
46             self._ensure_kernel(system.box)  
47             assert self.kernel is not None  
48  
49             short_result = self.short_range_solver.compute(system)  
50             forces = short_result.forces.copy()  
51             potential = short_result.potential  
52             virial = short_result.virial  
53  
54             positions = system.positions
```




```
52     charges = system.charges
53     n_particles = system.n_particles
54     m_samples = max(self.config.batch_size * n_particles, 1)
55
56     q_abs = np.abs(charges) + 1e-6
57     proposal = q_abs**self.config.importance_exponent
58     proposal /= np.sum(proposal)
59
60     i_all = rng.choice(n_particles, size=m_samples, p=
proposal)
61     j_all = rng.choice(n_particles, size=m_samples, p=
proposal)
62     mask_not_self = i_all != j_all
63
64     i_idx = i_all[mask_not_self]
65     j_idx = j_all[mask_not_self]
66     displacement = minimum_image(positions[j_idx] - positions
[i_idx], system.box)
67     r_sq = np.einsum("ij,ij->i", displacement, displacement)
68
69     cutoff_sq = self.config.cutoff_short * self.config.
cutoff_short
70     r_min_sq = self.config.r_min * self.config.r_min
71     mask_long = (r_sq > cutoff_sq) & (r_sq > r_min_sq)
72
73     i_valid = i_idx[mask_long]
74     j_valid = j_idx[mask_long]
75     disp_valid = displacement[mask_long]
76     r_sq_valid = r_sq[mask_long]
77
78     pair_prob = proposal[i_valid] * proposal[j_valid]
79     sample_weights = 1.0 / (m_samples * pair_prob)
80
81     pref = self.kernel.force_prefactor(r_sq_valid)
82     qij = charges[i_valid] * charges[j_valid]
83     pair_force = (qij * pref)[: , None] * disp_valid
84
```



```
85         scaled_force = sample_weights[:, None] * pair_force
86         np.add.at(forces, i_valid, scaled_force)
87         np.add.at(forces, j_valid, -scaled_force)
88
89         pair_potential = qij * self.kernel.evaluate_from_r_sq(
90             r_sq_valid)
91         potential += float(np.sum(0.5 * sample_weights *
92             pair_potential))
93         virial += float(np.sum(0.5 * sample_weights * np.einsum("
94             ij,ij->i", disp_valid, pair_force)))
95
96         return ForceResult(forces=forces, potential=potential,
97             virial=virial)
```

Listing 1 RBSOG 求解器真实核心代码节选

这一节选说明，正文中“随机批量采样”的实现应理解为“基于粒子电荷幅值的成对重要性采样”，而不是简单的均匀批量抽样。

A.3 近邻表与短程求解真实实现节选

下面代码节选自 `src/rbsog_md/forces/neighbor.py`，展示了 Verlet 近邻表的重建判据和短程库仑求解流程。

```
1 @dataclass
2 class VerletNeighborList:
3     cutoff: float
4     skin: float = 0.3
5     rebuild_interval: int = 10
6
7     @property
8     def pair_cutoff(self) -> float:
9         return self.cutoff + self.skin
10
11     def _needs_rebuild(self, system: ParticleSystem) -> bool:
12         if self._pairs is None or self._ref_positions is None or
13            self._ref_box is None:
14             return True
15         if not np.allclose(system.box, self._ref_box):
```



```
15         return True
16     if self._steps_since_build >= self.rebuild_interval:
17         return True
18     if self.skin <= 0.0:
19         return False
20
21     displacement = minimum_image(system.positions - self._ref_positions, system.box)
22     max_disp_sq = float(np.max(np.einsum("ij,ij->i", displacement, displacement)))
23     return max_disp_sq > (0.5 * self.skin) ** 2
24
25
26 class NeighborListCoulombSolver:
27     def compute(self, system: ParticleSystem, rng: np.random.Generator | None = None) -> ForceResult:
28         del rng
29         pairs = self.neighbor_list.get_pairs(system)
30         if pairs.size == 0:
31             return ForceResult(forces=np.zeros_like(system.positions), potential=0.0, virial=0.0)
32
33         return self._compute_vectorized(
34             positions=system.positions,
35             charges=system.charges,
36             box=system.box,
37             pairs=pairs,
38             cutoff_sq=self.cutoff * self.cutoff,
39             softening_sq=self.softening * self.softening,
40         )
```

Listing 2 近邻表与短程求解真实代码节选

A.4 SOG 核拟合真实实现节选

下面代码节选自仓库中的 SOG 核拟合实现文件：

src/rbsog_md/forces/sog.py



其核心是：在给定 $[r_{\min}, r_{\max}]$ 区间内，对 $1/r$ 进行最小二乘拟合，得到用于长程近似的高斯核参数。

```
1 def fit_sog_kernel(  
2     r_min: float,  
3     r_max: float,  
4     n_terms: int = 12,  
5     n_samples: int = 4096,  
6 ) -> SOGKernel:  
7     alpha_min = 0.5 / (r_max * r_max)  
8     alpha_max = 8.0 / (r_min * r_min)  
9     alphas = np.geomspace(alpha_min, alpha_max, n_terms)  
10  
11     r = np.geomspace(r_min, r_max, n_samples)  
12     design = np.exp(-np.outer(r * r, alphas))  
13     target = 1.0 / r  
14  
15     weights, _, _, _ = np.linalg.lstsq(design, target, rcond=None  
16 )  
17     weights = np.clip(weights, 0.0, None)  
18  
19     return SOGKernel(alphas=alphas, weights=weights, r_min=r_min,  
20                       r_max=r_max)
```

Listing 3 SOG 核拟合真实代码节选

A.5 复现实验命令

本文最终采用的实验流程与仓库中的命令行入口一致。对应协议可见 docs/experiment_protocol.md，核心命令如下：

```
1 # PPPM vs RBSOG 主基准  
2 rbsog-md benchmark \  
3     --solvers ppm, rbsog \  
4     --steps 1000 \  
5     --sample-interval 20 \  
6     --seeds 3 \  
7     --n-lipids 128 \  
8     --n-solvent 512 \  

```



```
9  --batch-size 100 \  
10 --grid 32 32 32 \  
11 --out results/thesis_final_benchmark_main  
12  
13 # 批量大小扫描  
14 rbsog-md sweep-batch \  
15   --steps 1000 \  
16   --sample-interval 20 \  
17   --seeds 3 \  
18   --batch-sizes 50,100,200,400 \  
19   --objective-time-weight 2 \  
20   --objective-variance-weight 1 \  
21   --out results/thesis_final_batch_sweep  
22  
23 # JIT 消融  
24 rbsog-md benchmark \  
25   --solvers rbsog \  
26   --steps 400 \  
27   --sample-interval 10 \  
28   --seeds 2 \  
29   --batch-size 100 \  
30   --out results/thesis_ablation_jit_on  
31  
32 rbsog-md benchmark \  
33   --solvers rbsog \  
34   --steps 400 \  
35   --sample-interval 10 \  
36   --seeds 2 \  
37   --batch-size 100 \  
38   --disable-numba \  
39   --out results/thesis_ablation_jit_off
```

Listing 4 论文核心实验命令



A.6 附录小结

附录的目的不是重复展示与正文无关的大段模板代码，而是说明论文中对 RBSOG、近邻表、SOG 核拟合以及实验流程的描述都能在当前仓库真实实现中找到对应依据。这样可以确保正文、附录、源码三者之间保持一致。



参考文献

- [1] ELLIS R J. Macromolecular crowding: obvious but underappreciated[J]. Trends in Biochemical Sciences, 2001, 26(10): 597–604.
- [2] MCCAMMON J A, GELIN B R, KARPLUS M. Dynamics of folded proteins[J]. Nature, 1977, 267(5612): 585–590.
- [3] LENNER N, et al. Whole-cell simulations: challenges and applications[J]. Annual Review of Biophysics, 2023, 52: 1–25.
- [4] FEIG M, et al. Current progress and future directions in molecular dynamics simulations of cells [J]. Journal of Chemical Theory and Computation, 2023, 19(15): 5001–5022.
- [5] SPITZER J, et al. Molecular dynamics simulations of cells[J]. Current Opinion in Structural Biology, 2023, 80: 102570.
- [6] ZHAO G, et al. Mature hiv-1 capsid structure by cryo-electron microscopy and all-atom molecular dynamics[J]. Nature, 2013, 497(7451): 643–646.
- [7] CASALINO L, et al. All-atom molecular dynamics simulations of the complete sars-cov-2 virion [J]. ACS Central Science, 2020, 6(11): 1983–1996.
- [8] DARDEN T, YORK D, PEDERSEN L. Particle mesh ewald: An $n \log n$ method for ewald sums in large systems[J]. The Journal of Chemical Physics, 1993, 98(12): 10089–10092.
- [9] HOCKNEY R W, EASTWOOD J W. Computer simulation using particles[M]. [S.l.]: CRC Press, 1988.
- [10] JIN S, LI L, LIU J G. Random batch methods (rbm) for interacting particle systems[J]. Journal of Computational Physics, 2020, 400: 108877.
- [11] GUO X, HE Y, WANG L L. Sum-of-gaussians tensor neural networks for high-dimensional schrödinger equation[J]. arXiv preprint arXiv:2508.10454, 2025.
- [12] ALLEN M P, TILDESLEY D J. Computer simulation of liquids[M]. 2nd ed. [S.l.]: Oxford University Press, 2017.
- [13] COHEN J. Statistical power analysis for the behavioral sciences[M]. 2nd ed. [S.l.]: Lawrence Erlbaum Associates, 1988.
- [14] DEB K, PRATAP A, AGARWAL S, et al. A fast and elitist multiobjective genetic algorithm: Nsga-ii[J]. IEEE Transactions on Evolutionary Computation, 2002, 6(2): 182–197.



致 谢

首先,我要衷心感谢我的指导教师 XXX 教授。在整个毕业设计过程中,XXX 教授给予了我悉心的指导和宝贵的支持。从课题选定、方案设计到实验分析、论文撰写,XXX 教授都给予了我耐心的指导和建设性的意见。XXX 教授严谨的治学态度、渊博的学识和敏锐的学术洞察力,使我受益匪浅,也为我今后的学习和工作树立了榜样。

感谢上海科技大学信息科学与技术学院的各位老师,他们在我的本科学习期间给予了我许多帮助和指导。特别感谢 XXX 老师在分子动力学模拟方面的专业指导,以及 XXX 老师在高性能计算方面的技术支持。

感谢课题组的各位同学,他们在实验过程中给予了我许多帮助和支持。特别感谢 XXX 同学在代码实现方面的贡献,以及 XXX 同学在数据分析方面的协助。

感谢上海科技大学提供的良好学习环境和丰富的学术资源,使我能够顺利完成本科学业和毕业设计。

感谢开源社区的众多贡献者,特别是 LaTeX 模板的开发者们。他们的辛勤付出和非凡工作,使得我能够使用 LaTeX 高效地撰写学位论文。特别感谢 ucasthesis 模板的开发者,以及 gbt7714-bibtex-style 的开发者,他们的工作为我的论文撰写提供了极大的便利。

感谢我的家人,他们在我求学期间给予了我无条件的支持和鼓励。他们的理解和关爱,是我前进的动力。

最后,感谢所有在我求学期间给予我帮助和支持的人。你们的善意和帮助,我将永远铭记在心。